Waipapa
Taumata Rau
University
of Auckland

CS340 – Tutorial Week 2

Getting started with Assignment 1

August 26

Connor Williams

About me

Plan for today

The aim for this tutorial is to get you to a point where you can start the assignment and feel comfortable making changes to the code yourself.

- Virtual Machines
- Getting comfortable in the terminal
- Setting up repositories
- Running xv6
- Making changes to the code

Attendance

Please scan the QR code and fill in your information.

E.g.

Name: Matt Smith

UPI: msmi851

Student ID: 556 434 323

Virtual Machines

Many ways to set up a VM.

Installation and set up varies on different hardware, and different operating systems.

Virtual Machines let you use and work in an operating system inside of a pre-existing operating system, while sharing system resources (CPU cores, memory, storage).

Plan for today

The aim for this tutorial is to get you to a point where you can start the assignment and feel comfortable making changes to the code yourself.

- Virtual Machines
- Getting comfortable in the terminal
- Setting up repositories
- Running xv6
- Making changes to the code

About tutorials

Students who regularly attend tutorials have a whopping 15% higher average course grade than those who don't!

Essential supplement to lectures.

We go over concepts used in assignments and tests/exams in great detail.

Tutorials finish with fun activities to reinforce your knowledge

Average Grades based on Tutorial Attendance

Average Grade for students with Tutorial Bonus Marks:
81.49069444% or A-

Average Grade for students with no Tutorial Bonus Marks:
69.03921% or B-

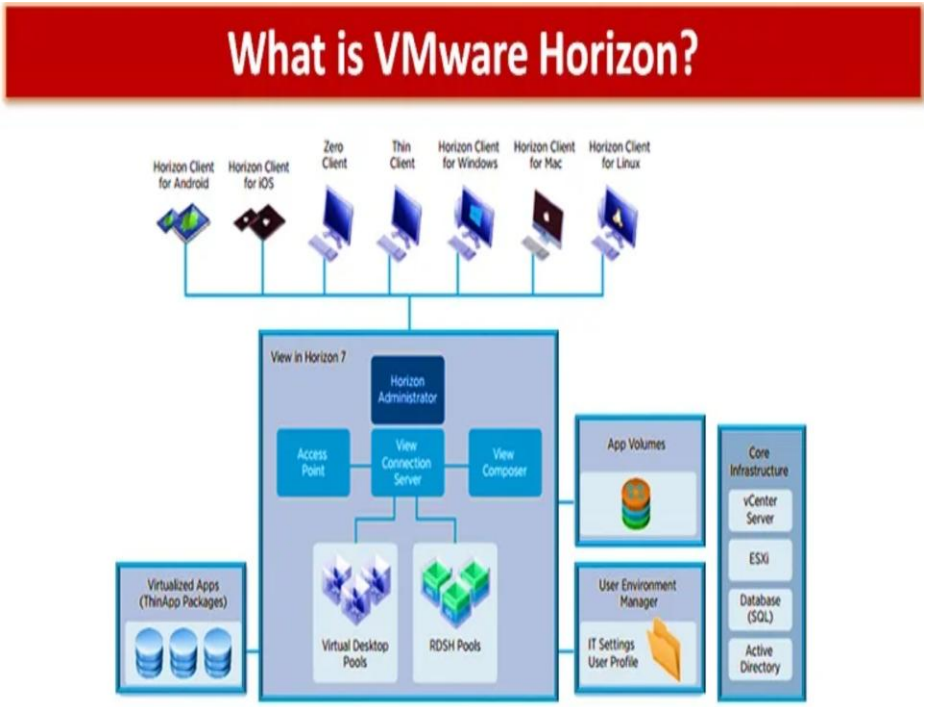
Part one:
Setting up

Getting Started

FlexIT is an online service that gives students access to software applications from any device at any time.

https://www.auckland.ac.nz/en/students/my-tools/flex-it/flexit-guide.html

You will need to **install the VMware Horizon Client** on your device.



Getting Started

FlexIT is an online service that gives students access to software applications from any device at any time.

https://www.auckland.ac.nz/en/students/my-tools/flex-it/flexit-guide.html

You will need to install the VMware Horizon Client on your device.

To install VMWare Horizon, open the below links, there is also a page in Canvas Modules that contains the download links:

Windows:
https://download3.omnissa.com/software/CART26FQ4_WIN_2512.1/Omnissa-Horizon-Client-2512-8.17.1-22261173697.exe

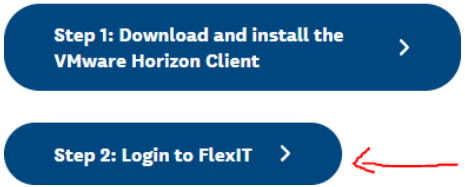
Mac:
https://download3.omnissa.com/software/CART26FQ4_MAC_2512.1/Omnissa-Horizon-Client-2512-8.17.1-22261165306.dmg

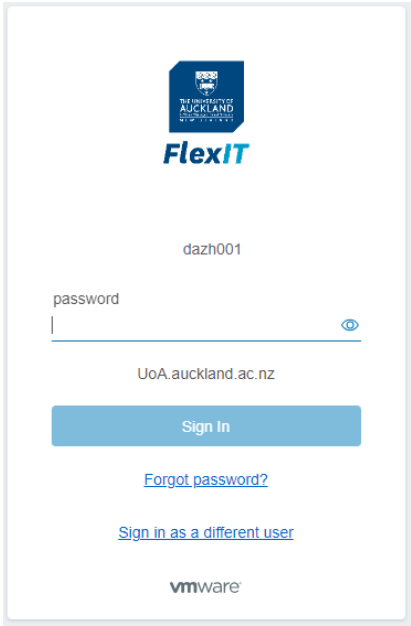
Linux:

https://download3.omnissa.com/software/CART26FQ4_LIN64_RPMPKG_2512.1/Omnissa-Horizon-Client-2512-8.17.1-22261155021.x64.rpm

Logging In

The next step is to log in to FlexIT using your University credentials.

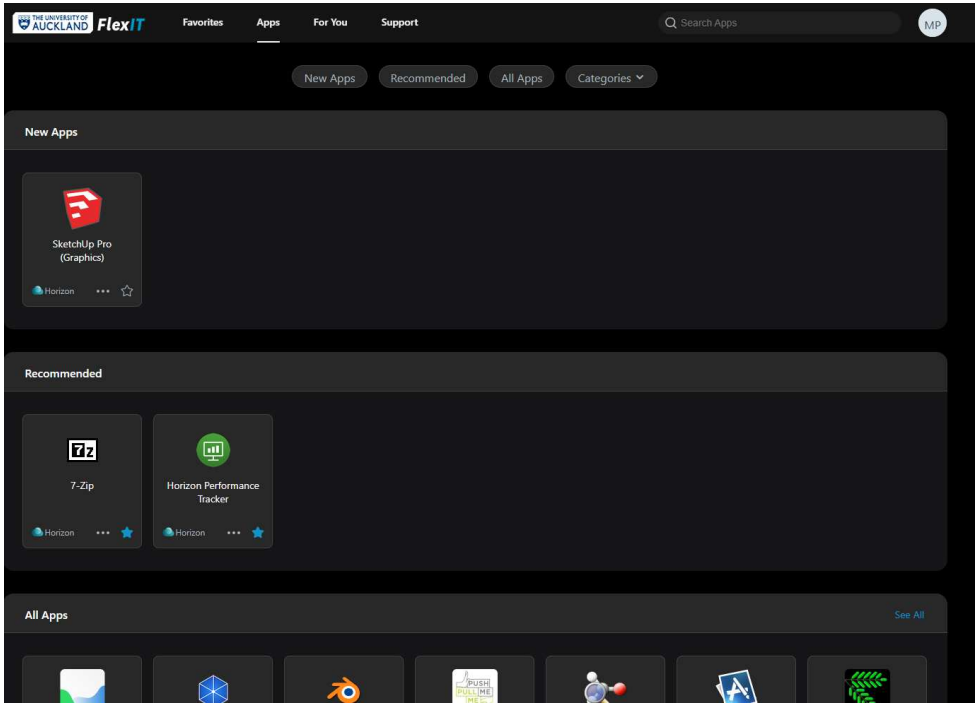




FlexIT Homepage

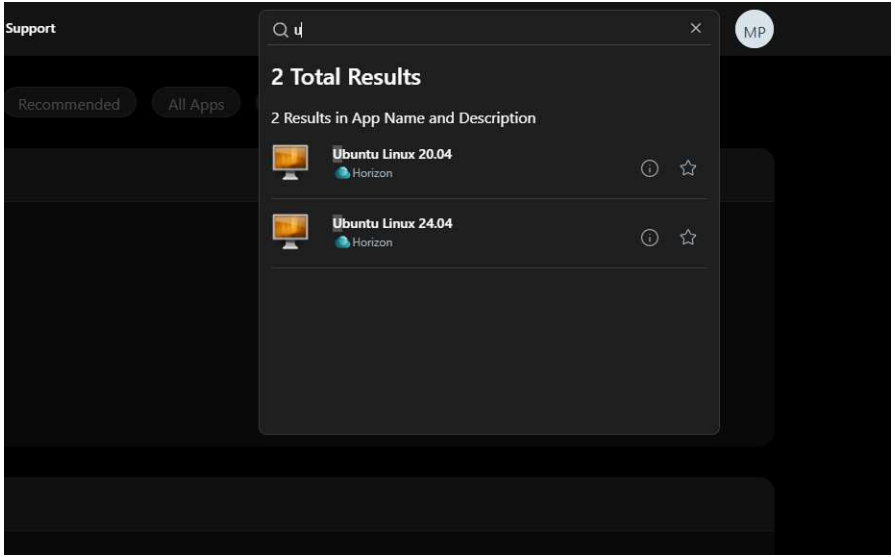
FlexIT gives you access to many useful applications that you can use from any computer at the University, at home, or anywhere!

One of these will be Ubuntu Linux 24.04 – a popular Linux distribution



Search for Ubuntu

1. **Search for Ubuntu Linux**
2. **Click ★ to add Ubuntu to Favourites (Optional)**
3. **Click Ubuntu Linux 24.04**



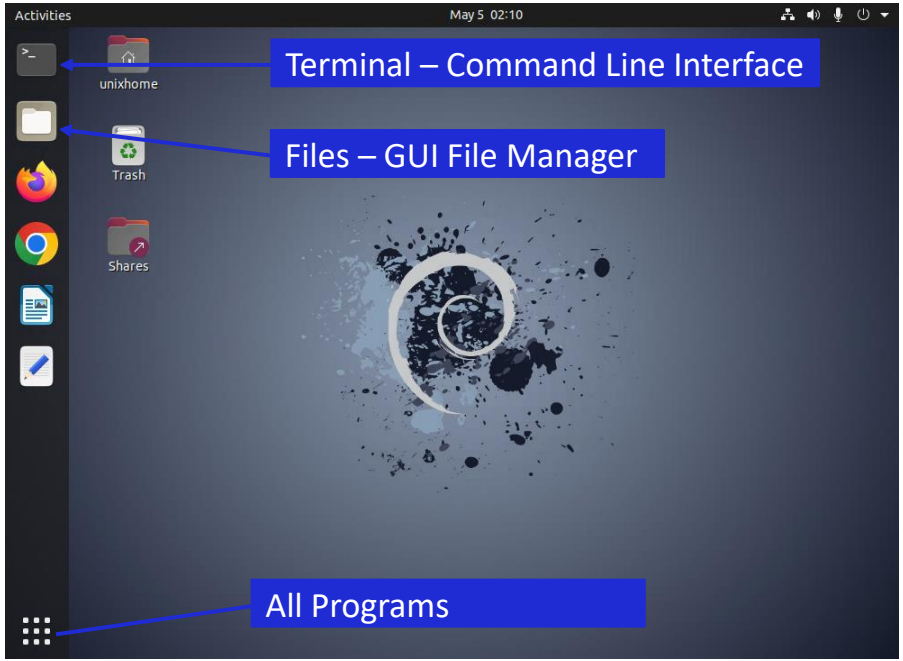
Ubuntu Homepage

Allow VMWare Horizon to connect to your local drive

You now have a Linux instance running on your computer!

You can access apps in the sidebar, all apps can be accessed by clicking the app drawer icon in the bottom left.

To open multiple instances of the same app, right-click the app icon, and click ‘New Window’

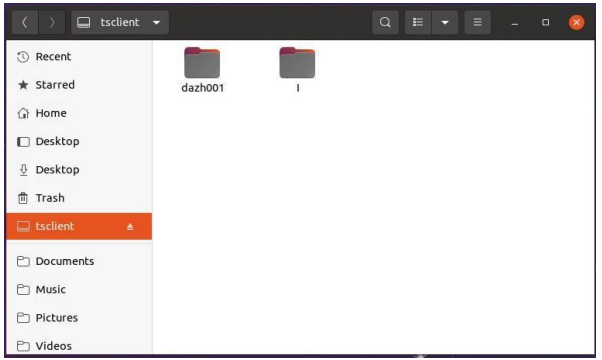


How To Connect A Local Drive

The first time you run Ubuntu on the VMware Horizon client, you may be prompted to connect a local drive.

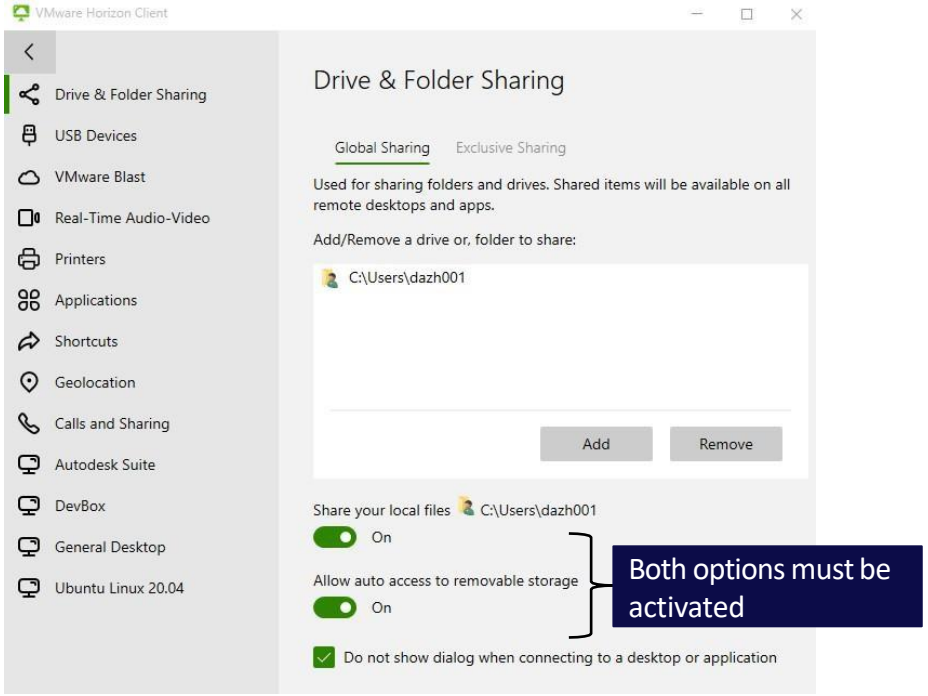
We recommend you do so as this will enable you to quickly and easily transfer files from your local machine to the Linux server and vice-versa.

- To access your local drive, open Files and click on “tsclient” to access your shared folders.
- You can then use the Files GUI to easily transfer files to/from your local machine and the Linux server.

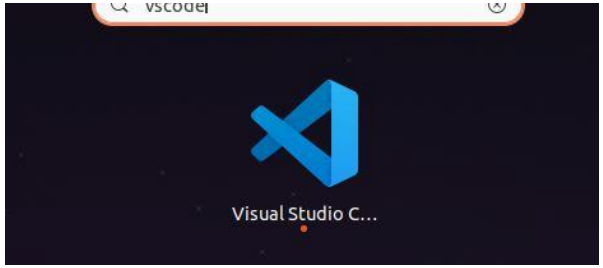


How To Connect A Local Drive

Step 3: Setup FlexIT to use your local drives



C Programming Environment



We will use Visual Studio Code in Labs, which is pre-installed on FlexIT Ubuntu Linux.

To find it, go to All Programs and find the icon in the list or search for it.

Once Visual Studio Code is installed, install the C/C++ Extension Pack from Microsoft

Part two:
Creating and Compiling C Programs

Hello, World! In C

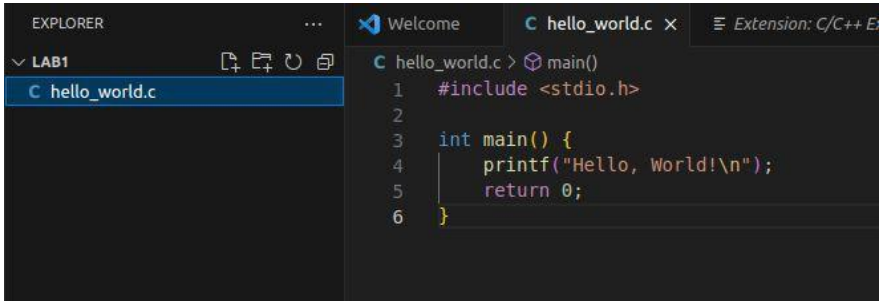
1. Create a new file named hello_world.c

   Ensure the file is created in the Linux file system, not tsclient

2. Add the template code for printing ‘Hello, World!’ to the console.
3. In the next slide, we will compile this code.

```
#include <stdio.h>

int main() {
    printf(“Hello,
    World!\n”);
    return 0;
}
```



Linux Terminal Commands

- **`ls` – lists the contents of the current directory**
  - **`ls -l` – gives a long listing**
  - **`ls -a` – shows hidden files and folders**
- **`pwd` – prints path of the current (working) directory**
  - **Ex: `/.../users/r/s/yourUPI/unixhome`**
- **`mkdir newDir` – make a new directory**

- **`cd ..`** – takes you back up the directory tree
- **`cd dirName`** – change directory to `dirName`.
  - Just **`cd`** will take you back to your home directory

- **`rm filename`** – delete (remove) a file
- **`rmdir dirName || rm -r dirName`** – delete a directory

- **`cp || mv`** – copies or moves files from one place to another

Compiling C Programs

```
zzha584@D2P1RB1Ubu203:~/Documents/lab1$ gcc -o hello_world hello_world.c
zzha584@D2P1RB1Ubu203:~/Documents/lab1$ ./hello_world
Hello, World!
zzha584@D2P1RB1Ubu203:~/Documents/lab1$
```

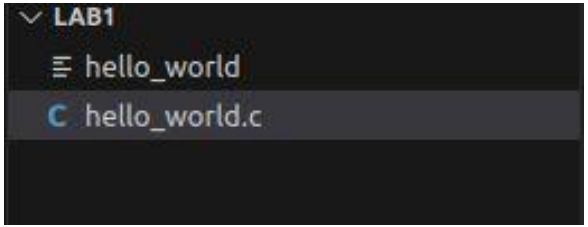
GCC is a C compiler that comes bundled with the Ubuntu distribution.

To compile programs, we type commands with the following syntax in the terminal.

gcc program.c –o output_filename

IMPORTANT: If you store your C files on tsclient (Host OS) then you will get errors here, to fix this, move your C files to your Ubuntu file system.

A good place to store files is in your home directory, in a COMPSCI 370 subfolder.



Upon compilation, a binary file will be created and shown in Visual Studio Code.

Part three:
Basic C Concepts

C Programming Basics

These are important foundational concepts!

Without mastering these, you will find that you cannot implement the assignments.

We will look at four core concepts that will be the foundation of your assignments.

1. **Arrays**
2. **Pointers**
3. **Structs**
4. **Dynamic Memory Allocation**

The man Command

For nicely formatted documentation you can visit:

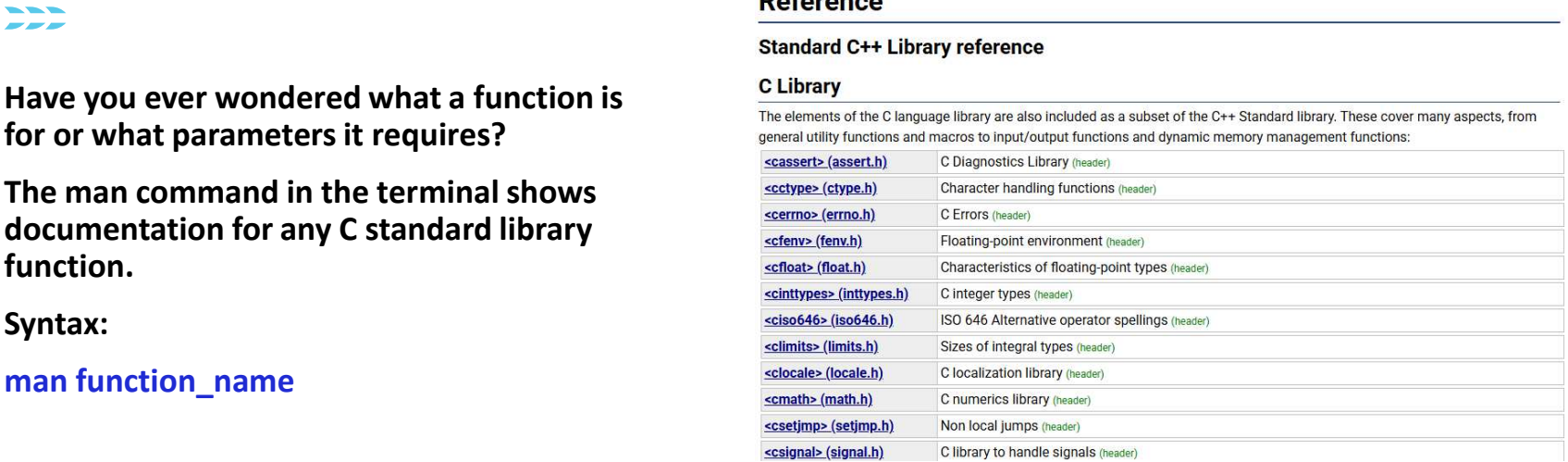
www.cplusplus.com/reference

Have you ever wondered what a function is for or what parameters it requires?

The man command in the terminal shows documentation for any C standard library function.

Syntax:

man function_name



Arrays

Arrays are fixed-size collections of a certain data type.

They use zero-based indexing and each element is stored contiguously in memory.

Syntax:

```
data_type  array_name[size];
```

- We can declare and initialise arrays in the same line:

```
int numbers[5] = {1, 2, 3, 4, 5};
```

- This initialises indexes 0 and 1, the rest hold **value 0**:

```
int numbers[5] = {1, 2};
```

- The compiler can also infer the array size for us.

```
int numbers[] = {1, 2, 3};
```

Arrays

Arrays are fixed-size collections of a certain data type.

They use zero-based indexing and each element is stored contiguously in memory.

Syntax:

```
data_type  array_name[size];
```

- We can declare multi-dimensional arrays.

```
int matrix[3][3] = {
   {1, 2, 3}, {4, 5, 6}, {7, 8, 9}
};
```

Array structure

	column 0	column 1	column 2
row 0	1	2	3
row 1	4	5	6
row 2	7	8	9

Example:

```
matrix[0][0] = 1
matrix[1][2] = 6
matrix[2][1] = 8
```

Pointers

Pointers are variables that store the memory address of another variable.

They allow direct access and modification of memory.

Syntax:

```
int x = 10;
int *p = &x;
```

Here, p points to the address of x

- Remember in C that pointers to arrays store the **base** memory location AND reference first item in the array

```
int[] array = {1, 2, 3, 4}
```

```
int *p = &array;
```

- `p` automatically points to the first element **1**

```
p += 1;
```

  - Now points to second element **2**

Structs

Structs are user-defined collections of objects of various types.

Typically used to model real-world entities.

Syntax:

```
struct StructName {
        data_type_1 member1;
        data_type_2 member2;
};
```

We can declare structs and initialise them like this:

```
struct Student {
        int id;
        char name[50];
};

struct Student s1 = {1, "Alice"};
```

We access struct members using **dot** notation:

```
printf("%s", s1.name);  // Output: Alice
```

However, if we have a pointer to a struct, we use **arrow** notation:

```
void updateStudent(struct Student *s) {
        s->id = 2;
}
```

Dynamic Memory Allocation

Memory can be allocated manually by the user

- **Allows for finer control of memory for better performance**
- **No automatic memory management in C**

Syntax:

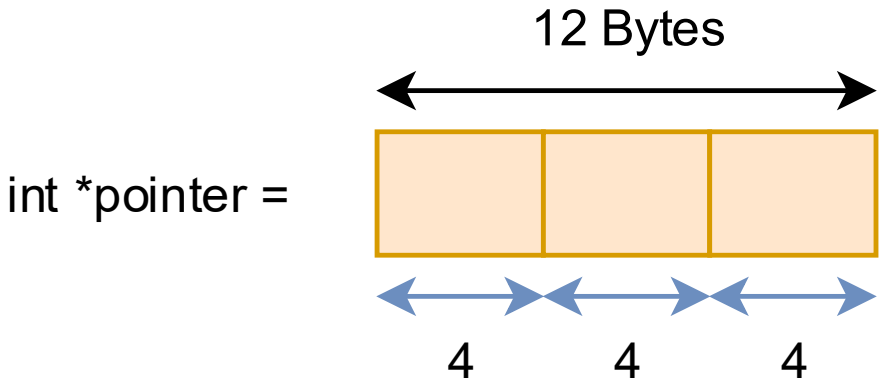
malloc(sizeof(data_type) * 2)

calloc(sizeof(data_type), 2)

- Both functions return a pointer to the memory location

int *pointer = (int*) malloc(sizeof(int) * 3)

- **Only calloc will initialise the memory locations to 0**



Thank you!

See you next week!